

Advanced Information Security

V0087/V0088

Instructor: Xiaohong Yu

Date: May 14, 2026

Due: May 28, 2026

Total Credits: 100 pts

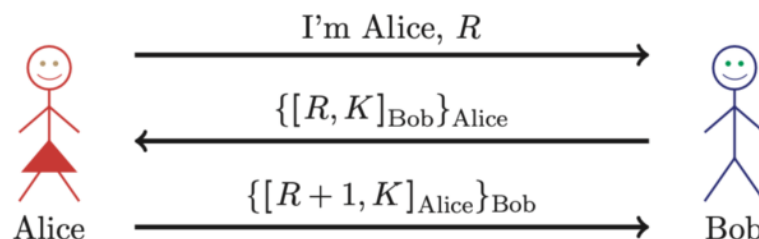
<Problem Solving>

1. 简单安全协议 [25points]

a. 已知一个“对称密钥双向认证”方案是把单向 challenge-response 协议直接执行两次来实现互认证。[10points]

- 1). 画出消息流程 (Alice、Bob、Trudy)。[3 points]
- 2). 说明 Trudy 如何利用反射攻击通过认证。指出该方案失败的根本原因。[3points]
- 3). 提出一种最小修改方案并解释为什么能防住反射攻击。[4 points]

b. 假设 Alice 和 Bob 打算通过一个不安全的网络建立一个共享的会话密钥 K , 以便后续进行加密通信。他们设计了如下图所示的认证与密钥协商协议。[15points]



- 1). 外层的公钥加密 (如 $\{...\}_{Alice}$) 主要提供了哪种安全属性? 保护了图中的哪个关键元素? [3points]

- 2). 内层的数字签名(如[...]_{Bob})主要提供了哪些安全属性? [3 points]
- 3). 仔细观察该协议, 它虽然使用了加密和签名, 但存在一个严重的逻辑漏洞。请写出当前协议可能遭受哪种攻击, 并描述攻击步骤。 [4 points]
- 4). 请给出一种方案修复上述漏洞。 [5 points]

2. 现实安全协议 [25points]

a. 早期的安全协议在设计时, 受限于当时的计算能力或对安全威胁的认知不足, 存在严重的设计缺陷。请分析以下三个早期协议/标准的致命弱点, 并说明现代协议是如何修复这些问题的: WEP (无线局域网保密协议), GSM (2G 移动通信网络) 和 SSL (安全套接字层协议) [10points]

b. 在过去的二十年里, 企业的标准安全架构是“边界防御”: 利用 IPsec VPN 将外网隔离, 员工接入 VPN 后, 在内网利用 Kerberos 实现单点登录 (SSO) 并自由访问各种服务。 [15points]

- 1). 在如今“云原生 + 居家办公”的时代, 为什么传统的“VPN + 内网 Kerberos”模型变得既不安全又低效? 请列举至少两个该模型在现代场景下的致命缺陷。 [5 points]
- 2). 请用一两句话概括“零信任 (Zero Trust)”架构的最核心原则。在零信任架构中, 传统的 IP 地址还被信任吗? 访问控制的决策主要基于什么? [5 points]
- 3). 随着业务迁移到公有云(如 AWS, Azure), 应用变成了无数的微服务和 Web API。在这种现代 Web 和云环境中, 由于跨域和互联网环境的限制, Kerberos 已经被边缘化。请写出目前工业界取代 Kerberos, 用于实现现代跨域单点登录 (SSO) 以及 API 授权授权的最主流的三个现代协议/标准框架名称, 并简单描述它们。 [5 points]

3. 软件安全 [20points]

a. 什么是‘金丝雀’技术? 其如何用于防止缓冲区溢出攻击? 微软是如何使用该技术的(编译器的 /GS 选项)? [10points]

b. 假如 Alice 的计算机设置了保护缓冲区溢出攻击的 NX 位。如果 Alice 的计算机中运行某些具有缓冲区溢出漏洞的程序，请问 Trudy 是否可以利用这些缓冲区溢出漏洞对 Alice 的计算机发动拒绝服务攻击？请说明原因。[10points]

4. 代码实践 [30points]

a. 下面是一段使用 AES-CBC 模式的 Python 伪代码/残缺代码。[15point]

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

key = os.urandom(32) # 256-bit key
iv = os.urandom(16) # 128-bit IV

def vulnerable_encrypt(plaintext_bytes):
    # (假设此处已包含对齐 padding 逻辑)
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    encryptor = cipher.encryptor()
    return encryptor.update(plaintext_bytes) + encryptor.finalize()

def vulnerable_decrypt(ciphertext_bytes):
    cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
    decryptor = cipher.decryptor()
    # 严重缺陷：解密后没有任何完整性校验就直接返回！
    return decryptor.update(ciphertext_bytes) + decryptor.finalize()
```

- 1). 请简述为什么上述 `vulnerable_decrypt` 函数是危险的？如果攻击者在网络上拦截了密文，随机修改了其中的几个字节再发送给接收方，接收方系统会发生什么？[5 points]
- 2). 请查阅 Python `cryptography` 库的官方文档，重写加密和解密函数。[10points]

要求：

- 废弃 AES-CBC，改用现代协议推荐的 **AES-GCM (Galois/Counter Mode)** 模式。
- 你的解密函数必须能够捕获并处理被篡改的密文（例如抛出 `InvalidTag` 异常），而不是返回错误的明文。
- 请提供完整的、可运行的 Python 源码。

b. 一位初级后端开发工程师为公司的内部系统编写了一个简单的登录验证模块。该模块使用 SQLite 数据库存储用户，并使用了 MD5 对用户密码进行“加密”存储。[15point]

```
import sqlite3
import hashlib

def init_db():
    conn = sqlite3.connect(':memory:') # 使用内存数据库方便测试
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE users (id INTEGER PRIMARY KEY, username TEXT, password_hash TEXT)")
    # 预置一个管理员账号，密码为 admin123 的 MD5 值
    cursor.execute("INSERT INTO users (username, password_hash) VALUES ('admin', '0192023a7bbd73250516f069df18b500')")
    conn.commit()
    return conn

def insecure_login(conn, username, password):
    # 开发者:我把密码 MD5 了一次，数据库里看不到明文，很安全！
    pwd_hash = hashlib.md5(password.encode()).hexdigest()

    # 开发者:直接拼接到 SQL 语句里查询
    query = f"SELECT * FROM users WHERE username='{username}' AND password_hash='{pwd_hash}'"

    cursor = conn.cursor()
    cursor.execute(query)
    user = cursor.fetchone()

    if user:
        print(f"登录成功！欢迎回来，{user[1]}。")
        return True
    else:
        print("登录失败：用户名或密码错误。")
        return False
```

- 1). 在不知道管理员密码的情况下，请构造一个恶意的 `username` 字符串（假设前端未做任何过滤）。当该字符串传入 `insecure_login` 时，能够触发 SQL 注入，直接绕过密码验证，打印出“登录成功”。请写出这个 payload。[5 points]
- 2). 修改 `insecure_login` 函数中的数据库查询逻辑。请使用参数化查询 (Parameterized Queries) 技术重写代码，彻底杜绝 SQL 注入漏洞。[10 points]

要求：

- 请提供完整的、可运行的 Python 源码